

PR Review Report:

`muozturk/dispatcher-te-parity`

Date: 2026-06-01

Branch: muozturk/dispatcher-te-parity → develop

Scope: projects/composablekernel/dispatcher/parity/ (14 new files, 2712 insertions)

Reviewed by: Three independent reviewers + requirements gap analysis against projectdes.txt

Table of Contents

- 1. [Executive Summary](#1-executive-summary)
- 2. [Reviewer 1 — Correctness & Logic](#2-reviewer-1--correctness--logic)
- 3. [Reviewer 2 — Code Quality & Maintainability](#3-reviewer-2--code-quality--maintainability)
- 4. [Reviewer 3 — Domain Expert (CK / GPU)](#4-reviewer-3--domain-expert-ck--gpu)
- 5. [Requirements Gap Analysis (projectdes.txt Phase 1)](#5-requirements-gap-analysis-projectdestxt-phase-1)
- 6. [Priority Action List](#6-priority-action-list)

1. Executive Summary

This PR adds a parity test suite that proves the **CK Tile Dispatcher** reproduces the **Tile Engine** for GEMM: same kernel, same registry key, same numerical output, and (within tolerance) the same throughput. The suite is split into CPU-only stages (identifier parity, codegen) and GPU-gated stages (numerical, performance).

Files added:

File	Lines	Role
check_parity.py	528	Main orchestrator
make_docs.py	517	PDF documentation generator
te_to_dispatcher.py	367	TE config → dispatcher config translator
check_identifier_parity.py	183	Python vs C++ identifier oracle
drive_codegen.py	183	Single-config codegen driver
identifier.py	123	Python encode_identifier mirror
harness.cpp	182	GPU benchmark/verify harness
cpp_identifier_oracle.cpp	136	C++ identifier oracle (g++ only)
README.md	127	Design overview
build_harness.sh	56	Harness build script
configs/single_fp16_rcr.json	27	Example config
.gitignore	10	Ignores built binaries

File	Lines	Role
parity_design.pdf	—	Committed binary
parity_usage.pdf	—	Committed binary

Overall verdicts:

Reviewer	Verdict
Correctness & Logic	Request Changes
Code Quality & Maintainability	Request Changes
Domain Expert (CK / GPU)	Request Changes

Requirements coverage (vs projectdes.txt Phase 1):

Task	Coverage	Risk
T1.1 Config Translator	75%	Medium
T1.2 Kernel Name Round-trip	75%	Low
T1.3 Drive Codegen	75%	Low
T1.4 Minimal Harness	85%	Low
T1.5 Parity Checker	55%	High
T1.6 Numerical Parity	60%	High
T1.7 Performance Parity	40%	High

2. Reviewer 1 — Correctness & Logic

Verdict: Request Changes

2.1 Bugs

Bug 1 — `te_kernel_name()` **missing** `_preshuffle` **suffix** (Severity: High)

`check_parity.py` lines 87–93 construct the generated header filename without a `_preshuffle` suffix. However, `unified_gemm_codegen.py`'s `KernelNaming.generate()` appends `_preshuffle` for `GemmVariant.PRESHUFFLE` configs. The result: for any `preshufflev2` config, Stage 2 exits immediately with "expected generated header not found." Stages 2 and 3 are silently broken for all `preshuffle` configs.

Bug 2 — `_minimal_te_config()` **drops** `block_size`, `num_wave_groups`, `k_block_per_cu` (Severity: High)

`drive_codegen.py`'s `_minimal_te_config()` does not forward `block_size`, `num_wave_groups`, or `k_block_per_cu` from the TE config to `unified_gemm_codegen`. The codegen silently defaults to `block_size=256`, `num_wave_groups=1`, `k_block_per_cu=1`. Any TE config with non-default values produces a silently wrong kernel. The bundled `single_fp16_rcr.json` happens to use the defaults, masking the bug.

Bug 3 — `_PIPELINE_CANON` **accepts unsupported pipelines** (Severity: Medium)

`te_to_dispatcher.py`'s `_PIPELINE_CANON` includes `compv1`, `compv2`, and `preshufflev1`. These have no path in `codegen_common.PIPELINE_TO_DISPATCHER` and `unified_gemm_codegen` has no codegen logic for them. A TE config specifying these passes translation silently, then fails at codegen with an opaque error. A `TranslationError` should

be raised at translation time.

Bug 4 — Dry-run temp file leak (Severity: Low)

drive_codegen.py creates a NamedTemporaryFile(delete=False) (line ~107). The finally block cleans it up on the normal path, but on --dry-run (lines 132–134), it prints the path and returns without deleting. Every dry-run invocation leaks a JSON file in /tmp.

Bug 5 — double_buffer discrepancy for preshufflelev2 (Severity: Medium)

te_to_dispatcher.py: _DOUBLE_BUFFER_PIPELINES = {"compv4", "preshufflelev2"} → double_buffer=True for preshufflelev2.

unified_gemm_codegen.py line 831: key.algorithm.double_buffer = {str(config.trait.pipeline == "compv4").lower()} → double_buffer=False for preshufflelev2.

double_buffer is in tie() (equality) but NOT in encode_identifier(), so Stage 1 passes but KernelKey::operator== gives the wrong answer. Pre-existing in codegen; out of scope to fix here, but must be filed as a follow-up.

2.2 Edge Cases Missed

- **split_k > 255:** cpp_identifier_oracle.cpp line 68 casts to uint8_t; 256 wraps to 0, causing Python/C++ mismatch. No range validation anywhere in the pipeline.
- **Harness rcr-only strides:** harness.cpp hardcodes stride_a=K, stride_b=K, stride_c=N (rcr layout) but check_parity.py never validates the config layout is actually rcr before building/running the harness.
- **Dry-run identifier stage:** stage_identifier() returns True immediately when dry_run=True (line ~330), so the summary shows PASS despite nothing being checked.
- **stderr discarded from GPU processes:** run_harness() and run_te_benchmark() never forward proc.stderr. HIP errors (OOM, device not found) are silently swallowed; verdict is UNKNOWN with no diagnostic.

2.3 Error Handling Gaps

- No timeouts on codegen, build, or GPU execution subprocesses (only rocminfo has 30 s).
- te_*.csv output files are not gitignored — accumulate after every GPU run.
- No unit tests for any Python translation logic (te_to_dispatcher.translate(), encode_identifier(), etc.).

3. Reviewer 2 — Code Quality & Maintainability

Verdict: Request Changes

3.1 Structural Issues

Six repeated _print_summary(); return 1 **idioms** (check_parity.py lines 332–333, 347–348, 360–361, 368–369, 374–375, 406–407). Extract a _fail(summary, **stages) -> int helper.

stderr **discarded from GPU subprocesses** — run_harness and run_te_benchmark never write proc.stderr. All other callers (build_harness, drive_codegen, stage_identifier) correctly forward stderr.

"DRYRUN" **sentinel is an undocumented internal contract** — run_harness returns {"verdict": "DRYRUN"} when dry_run=True. _adjudicate_numerical checks for this string at line 449. No module-level constant, no comment.

Type annotations missing on _adjudicate_numerical and _adjudicate_performance, inconsistent with every other function in the file.

3.2 Naming & Clarity

Location	Issue	Suggestion
check_parity.py line ~85	cap = lambda b: ... (PEP 8: no named lambdas)	Use def _capitalize_bool(b)
check_parity.py	te_active means "did we find the TE executable"	Rename to has_te_exe
check_parity.py	parse_te_csv's _f helper	Rename to _column_float
check_parity.py	"=" * 72 repeated 9 times	Extract_SEP = "=" * 72
te_to_dispatcher.py	_LAYOUT_CHAR["p"] looks like a no-op identity map	Add comment: # p = PackedExternal

3.3 Code Duplication

- translate_file() + bounds-check logic is duplicated in check_parity.py (lines 302–310) and drive_codegen.py (lines 88–94).
- "No valid dispatcher configs" error string is printed identically in check_identifier_parity.py:144, check_parity.py:304, drive_codegen.py:90.
- iter_configs() in te_to_dispatcher.py (lines 337–338) is dead code — never imported or called anywhere.

3.4 Dead Code & Hygiene

- parity_design.pdf and parity_usage.pdf are committed to git. Binary artifacts generated by make_docs.py produce noisy diffs on regeneration. Built binaries (harness, cpp_identifier_oracle) are gitignored but the generated PDFs are not — inconsistent policy.
- te_*.csv files written by run_te_benchmark are not gitignored.
- te_to_dispatcher.py:main() uses a lazy from identifier import encode_identifier (line 359). The comment says "keep translate() dependency-free" — correct reasoning but the comment should live on main(), not translate().

3.5 README Issues

- "What runs where" table labels both "Numerical parity" and "Performance parity" rows as (f) — should be (e) and (f).
- No warning that te_*.csv files accumulate in the parity directory after GPU runs.

4. Reviewer 3 — Domain Expert (CK / GPU)

Verdict: Request Changes

4.1 Translation Correctness

Verified field-by-field against kernel_key.hpp, codegen_common.py, unified_gemm_codegen.py:

Field	Status
Accumulation dtype (fp16→fp32, bf16→fp32, fp8→fp32, int8→int32)	Correct
Output dtype promotion (fp8/bf8→fp16)	Correct
Scheduler canonicalization (default→auto)	Correct
Pipeline canon table (8 entries)	Correct for supported pipelines
_UNSUPPORTED_TRAITS set	Matches codegen exactly
_Tile.is_valid()	Byte-for-byte equivalent to TileConfig.is_valid()
transpose_a/b/c, grouped all hardcoded False	Correct for standard non-transposed GEMM
double_buffer for preshufflev2	MISMATCH — translator: True, codegen: False

The double_buffer discrepancy: translator sets True for both compv4 and preshufflev2; codegen line 831 sets True only for compv4. This does not affect encode_identifier() (which omits double_buffer) but breaks KernelKey::operator== for preshufflev2 keys. The bundled config uses compv3 so this is not currently exercised.

4.2 Identifier Parity Soundness

The two-oracle approach (Python encode_identifier vs C++ KernelKey::encode_identifier, compared byte-for-byte, compiled from the actual runtime header with plain g++) is the correct design. All fields verified:

Field	Python	C++	Match
dtype_a, layout_a/b/c	sig['dtype_a'], sig['layout_*']	to_string(sig.dtype_a), to_string×3	Yes
pipeline, epilogue, scheduler	alg['pipeline']_, etc.	to_string(alg.*)_	Yes
pad_m/n/k, persistent	"True"/"False"	(flag ? "True" : "False")	Yes
tile/warp/warp_tile	{m}×{n}×{k}_...	Identical format	Yes
splitk suffix	_splitk{n} if >1	Same	Yes
elementwise_op suffix	Skips empty/PassThrough	Same	Yes
sparse/preshuffle suffix	_sparse, _preshuffle	Same	Yes

Omission of block_size and gfx_arch from the identifier is correct — C++ encode_identifier() also omits both.

4.3 Numerical Parity Strategy

Tolerance formula: $1e-2 * \sqrt{K}$ applied to max_abs_err only.

This is too loose for the input scale used. The harness initializes values in $[-0.75, 0.75]$ (A) and $[-0.5, 0.5]$ (B). Theoretical fp16 stochastic rounding error for a $K=512$ dot product: $\sqrt{512} \cdot 1e-3 \cdot 0.56 \approx 0.013$. The tolerance at $K=512$: $1e-2 * \sqrt{512} \approx 0.226$. A kernel returning values off by 0.2 would pass unchallenged.

Recommendation: Gate on $\max_rel_err < 1e-2$ (relative) or tighten absolute constant to $1e-3$.

Stride setup: Correct for rcr. stride_a=K (row-major A), stride_b=K (col-major B, $B[k,n]=ptr[n*K+k]$), stride_c=N. CPU reference indexing in harness.cpp is consistent.

Performance benchmarking: nrepeat_=20 with 3 warmup iterations is reasonable for a parity check.

4.4 Test Coverage Gaps

Gap	Impact
Only compv3 pipeline tested	compv4 (double-buffer), preshufflelev2 (preshuffle) — historically most divergence-prone — not exercised
All sizes tile-aligned (512/1024/2048 divisible by 256/128/32)	Padding code path (pad_m/n/k=True) never triggered
No bf16, int8, cshuffle epilogue configs	Different dtype/epilogue paths unverified
No split-K or grouped config	Identifier suffix paths present but never exercised
Harness hardcoded rcr; no other layouts in config set	ccr, rcc, rrr variants untested
Stage 3 requires --te-build-dir	Cannot run standalone; always shows INFO in CI without co-located TE build

5. Requirements Gap Analysis (projectdes.txt Phase 1)

The project description (projectdes.txt) defines seven tasks for Phase 1. This section evaluates how completely each is satisfied.

T1.1 — Config Translator — 75% — Medium Risk

Spec "done" criteria: Unit tests for vanilla fp16 rcr, padding-enabled, split_k>1, persistent kernels.

What's implemented:

- Warp/wave naming handled correctly (warp_m/n/k in TE JSON → wave_shape.m/n/k in C++ struct via oracle)
- All fields emitted explicitly, no defaults relied on (pad_m/n/k, persistent, double_buffer, etc.)
- Enum/string canonicalization tables for pipeline, scheduler, epilogue
- split_k field read and emitted

What's missing:

- **No unit tests at all.** No test_te_to_dispatcher.py or equivalent exists anywhere.
- No config files for: padding=true, split_k>1, persistent=true. Only single_fp16_rcr.json (all defaults).
- Warp→wave naming trap not commented in te_to_dispatcher.py source.

T1.2 — Kernel Name Round-trip — 75% — Low Risk

Spec "done" criteria: Single command loops over handful of configs; any mismatch fails loudly.

What's implemented:

- Python encode_identifier vs C++ oracle comparison, verified field-by-field
- Batch processing via stdin (--- separator); single process for N configs
- Exit code 1 on any mismatch; each mismatch printed with [FAIL] label
- README reports 283,968/283,968 configs matched on a multi-combo file

What's missing:

- Bundled test suite has only 1 config (single_fp16_rcr.json → 1 combination). The "handful of configs" is only available if user provides a multi-value config externally.
- No verification that codegen output filename matches the identifier (the third leg of the round-trip).

T1.3 — Drive Codegen — 75% — Low Risk

Spec "done" criteria: Run codegen with JSON → exact expected files in chosen directory, no extra kernels.

What's implemented:

- drive_codegen.py translates TE JSON, selects one config by --index, invokes unified_gemm_codegen.py
- Prints generated headers and registry identifier after run

What's missing:

- drive_codegen.py never asserts len(generated_headers) == 1; just prints the list.
- Dry-run temp file leak.
- Master registration header interaction not documented.

T1.4 — Minimal Harness — 85% — Low Risk

Spec "done" criteria: Builds with one command; runs to completion; prints "ran kernel X on (M,N,K) in N ms"; no asserts; plausible output.

What's implemented:

- Allocates A/B/C, deterministic init (i%7-3)*0.25, constructs GemmHostArgs, runs SelectedKernel::launch, copies result, prints timing
- Builds with build_harness.sh (one command)
- try/catch around launch — unsupported configs print SKIPPED, not crash

What's missing:

- Output format mismatch: harness prints "kernel : X" and "time : Y ms" on separate lines rather than spec format "ran kernel X on (M,N,K) in N ms".
- cold_niters_=3 warmup is the struct default, never explicitly set or documented.

T1.5 — Parity Checker — 55% — HIGH Risk

Spec "done" criteria: Invokes both stacks; identical initialization; max abs/rel error + % within tolerance; first 10 mismatches on failure.

What's implemented:

- Invokes dispatcher harness and optionally TE benchmark
- Parses verdict from stdout
- Adjudicates pass/fail per size

What's missing (critical):

CRITICAL: Initialization is NOT identical between TE and dispatcher. harness.cpp uses (i%7-3)*0.25 (fixed pattern). TE benchmark defaults to FillUniformDistribution{-1,1} (random). run_te_benchmark in check_parity.py passes no init flag to force a specific pattern. The two stacks see **different data**. Each verifies against its own CPU reference — they are checking self-consistency, not cross-stack agreement. A bug where both stacks implement the same wrong computation would pass.

Additional gaps:

- No % within tolerance computation (harness only tracks max, not per-element pass rate)
- No "first 10 mismatches on failure" (harness prints max_abs_err only)
- Tolerance formula $1e-2 \cdot \sqrt{k}$ (abs only) differs from spec $atol + rtol * |ref|$ (element-wise relative)

T1.6 — Numerical Parity — 60% — HIGH Risk

Spec "done" criteria: PASS on ordinary size AND on awkward non-tile-aligned size (e.g., 257×257×257).

What's implemented:

- Default --sizes includes 1024x1024x1024 (ordinary size)

- Numerical-before-performance stage ordering enforced

What's missing:

- **No non-tile-aligned size in defaults.** Sizes 512/1024/2048 are all divisible by tile_m=256, tile_n=128, tile_k=32. No padding code path is triggered.
- **No padding-enabled config.** single_fp16_rcr.json has pad_m/n/k=false. A 257-size problem would be SKIPPED (not FAILED) because the kernel's supports() predicate rejects it.
- Parity on GPU not yet demonstrated (README explicitly notes numerical is SKIPPED on the development box).
- Tolerance loosening (if any occurred) not documented with justification.

T1.7 — Performance Parity — 40% — HIGH Risk

Spec "done" criteria: Both stacks within ~2% mean time across 10 back-to-back runs; methodology documented.

What's implemented:

- Stage 3 compares dispatcher TFLOP/s vs TE TFLOP/s with --perf-tol (default 10%)
- Harness sets nrepeat_=20 (≥10 iterations)

What's missing:

- **Mean ≠ median.** Spec requires comparing medians across 10 independent runs. Harness returns mean of 20 repeats in a single launch; run_harness has no outer loop for independent invocations.
- **TE timing not reconciled.** run_te_benchmark passes no --warmup or --repeat flags to the TE executable, which uses its own defaults. Comparing different warmup/repeat settings is exactly the false-parity scenario the spec warns about.
- **Tolerance 10% vs spec 2%.** The default --perf-tol=0.10 is 5× looser than the spec's ~2%. No comment explains the deviation.
- **No methodology documentation.** Warmup count, cache-flush strategy, GPU boost state handling, tolerance rationale — none documented.
- **Not demonstrated on GPU.**

6. Priority Action List

Must Fix Before Merge

#	Location	Issue
1	check_parity.py te_kernel_name()	Add _preshuffle suffix for preshuffle variants
2	drive_codegen.py _minimal_te_config()	Forward block_size, num_wave_groups, k_block_per_cu
3	check_parity.py run_te_benchmark()	Pass identical fixed-pattern init flag to TE benchmark; compare TE and dispatcher outputs on same data
4	harness.cpp line 164	Tighten numerical tolerance: gate on max_rel_err < 1e-2 or reduce abs constant to 1e-3
5	harness.cpp	Add --layout guard: assert config layout is rcr before running (or generalize strides)

Should Fix Before Merge

#	Location	Issue
6	check_parity.py default --sizes	Add non-tile-aligned size (e.g., 513x511x33)
7	configs/	Add single_fp16_rcr_pad.json (pad_m/n/k=true) for padding code path
8	check_parity.py run_te_benchmark()	Pass --warmup 3 --repeat 20 to match harness
9	te_to_dispatcher.py	Add _PIPELINE_CANON guard: raise TranslationError for compv1, compv2, preshufflev1
10	check_parity.py	Fix 6x _print_summary(); return 1 with _fail() helper
11	.gitignore	Add te_*.csv

Can Be Follow-ups

#	Issue
12	Write unit tests for te_to_dispatcher.translate() (vanilla, padding, split_k, persistent)
13	Add "handful of configs" JSON to configs/ for T1.2 coverage
14	T1.7: document timing methodology; decide 10% vs 2% tolerance with justification
15	Forward stderr from run_harness and run_te_benchmark
16	File issue: double_buffer discrepancy for preshufflev2 in unified_gemm_codegen.py
17	Gitignore parity_design.pdf, parity_usage.pdf (or policy-document that they are intentionally committed)
18	Dead code: remove iter_configs() from te_to_dispatcher.py
19	Fix README table: (f)/(f) → (e)/(f) for numerical/performance parity rows
20	Fix dry-run temp file leak in drive_codegen.py

Report generated 2026-06-01. Reviewed by three independent agents analyzing the diff of branch muozturk/dispatcher-te-parity against develop.